



DIPLOMADO

Desarrollo de sistemas con tecnología Java

Módulo 5

Desarrollo de aplicaciones empresariales con Jakarta EE

Uriel Hernández



API Usuario

Registrar Usuario

Usuario

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Entity
public class Usuario {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Integer id;
    private String nombre;
    @Column(name="primer_apellido")
    private String primerApellido;
    @Column(name="segundo_apellido")
    private String segundoApellido;
    private String password;
    private String email;
    private String rfc;
}
```

JpaUsuarioRepository

```
@Singleton
public class JpaUsuarioRepository implements UsuarioRepository {

    @PersistenceContext(unitName="pixup")
    private EntityManager entityManager;

    @Override
    public Optional<Usuario> findByEmail(String email) {
        TypedQuery<Usuario> query = entityManager.createQuery(
            "SELECT u FROM Usuario u WHERE u.email = :email", Usuario.class);
        query.setParameter("email", email);
        List<Usuario> usuarios = query.getResultList();
        return !usuarios.isEmpty() ? Optional.of(usuarios.get(0)) : Optional.empty();
    }

    @Override
    public Usuario save(Usuario usuario) {
        entityManager.persist(usuario);
        return usuario;
    }
}
```

UsuarioService

```
@Local
public interface UsuarioService {

    Usuario registrarUsuario(Usuario usuario, Domicilio domicilio);

}
```

UsuarioServiceImpl

```
@Stateless
public class UsuarioServiceImpl implements UsuarioService {

    @Inject
    private UsuarioRepository usuarioRepository;
    @Inject
    private DomicilioRepository domicilioRepository;
    @Inject
    private ColoniaRepository coloniaRepository;
    @Inject
    private TipoDomicilioRepository tipoDomicilioRepository;

    @Override
    @Transactional(value=Transactional.TxType.REQUIRED)
    public Usuario registrarUsuario(Usuario usuario, Domicilio domicilio) {
        // validacion usuario duplicado
        Optional<Usuario> usuarioExistente =
            usuarioRepository.findByEmail(usuario.getEmail());
        if (usuarioExistente.isPresent()) {
            throw new UsuarioAlreadyExistsException(usuario.getEmail());
        }
    }
}
```

JTA (Java Transaction API)

- Es una especificación Java que define un estándar de interfaces entre un gestor de transacciones (transaction manager) y las partes involucradas en un sistema de transacciones distribuidas.
- Permite a las aplicaciones realizar transacciones distribuidas por medio del contenedor (application server). Las aplicaciones pueden definir las transacciones de forma declarativa por medio de la anotación **@Transactional**

Propagation Behaviour (Comportamiento de propagación)

- Se refiere al comportamiento que debe tener la ejecución de un método en un contexto transaccional, existen los siguientes comportamientos:
 - **REQUIRED:** Debe invocarse dentro de una transacción, si existe una transacción en curso se el método se ejecutará dentro de esa transacción, si no existe una transacción en curso se creará una nueva.
 - **MANDATORY:** Debe invocarse dentro de una transacción, si no existe una transacción en curso se generará una excepción.
 - **NEVER:** No debe invocarse dentro de una transacción, si existe una transacción en curso se generará una excepción.
 - **NOT_SUPPORTED:** No debería participar dentro de una transacción, si existe una transacción en curso se suspende temporalmente, reanuda la ejecución de la transacción cuando acaba la ejecución del método.
 - **REQUIRES_NEW:** Siempre inicia una nueva transacción incluso si una transacción en curso existe.
 - **SUPPORTS:** Permite participar al método dentro una transacción si es que existe, si no existe una transacción en curso no se crea una nueva.

ACID

- Conjunto de propiedades que deben exhibir las transacciones de base de datos para garantizar la validez de los datos a pesar de los errores.
- **Atomicity:** Cada transacción (conjunto de operaciones) es tratada como una unidad, por lo que se ejecuta todo el conjunto de manera exitosa (commit) o en caso de error no se aplica ninguna operación (rollback).
- **Consistency:** Una transacción sólo puede llevar a la BD de un estado consistente a otro, cada dato escrito a la BD debe ser válido de acuerdo a los constraints, cascades, triggers e integridad referencial.
- **Isolation:** Las transacciones se ejecutan de manera concurrente (múltiples transacciones pueden estar leyendo/escribiendo en una tabla al mismo tiempo). El nivel de aislamiento define el comportamiento que sucederá entre transacciones (por ejemplo: si una transacción puede ver la modificación de datos realizada por otra transacción que aún no ha dado commit).
- **Durability:** Las transacciones completadas son almacenadas en memoria no volátil (discos duros).



Práctica No. 3

RegistroUsuarioDTO

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class RegistroUsuarioDTO {

    @NotNull(message="Usuario es requerido para realizar el registro")
    @Valid
    private UsuarioRequestDTO usuario;

    @NotNull(message="Domicilio es requerido para realizar el registro")
    @Valid
    private DomicilioDTO domicilio;

}
```

UsuarioApi

```
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
@Path("usuarios")
public interface UsuarioApi {

    @POST
    @Path("registro")
    Response registrarUsuario(
        @NotNull @Valid RegistroUsuarioDTO registroUsuarioDTO);
}
```

UsuarioResource

```
public class UsuarioResource implements UsuarioApi {  
  
    @Inject  
    private RegistroUsuarioMapper registroUsuarioMapper;  
    @Inject  
    private UsuarioService usuarioService;  
  
    @Override  
    public Response registrarUsuario(RegistroUsuarioDTO registroUsuarioDTO) {  
        Usuario usuarioCreado = usuarioService.registrarUsuario(  
            registroUsuarioMapper.toUsuario(registroUsuarioDTO.getUsuario()),  
            registroUsuarioMapper.toDomicilio(registroUsuarioDTO.getDomicilio()));  
        return Response.status(Response.Status.CREATED)  
            .entity(registroUsuarioMapper.toDto(usuarioCreado)).build();  
    }  
}
```

ErrorResponse

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class ErrorResponse {

    private Integer estatus;
    private String tipo;
    private String mensaje;

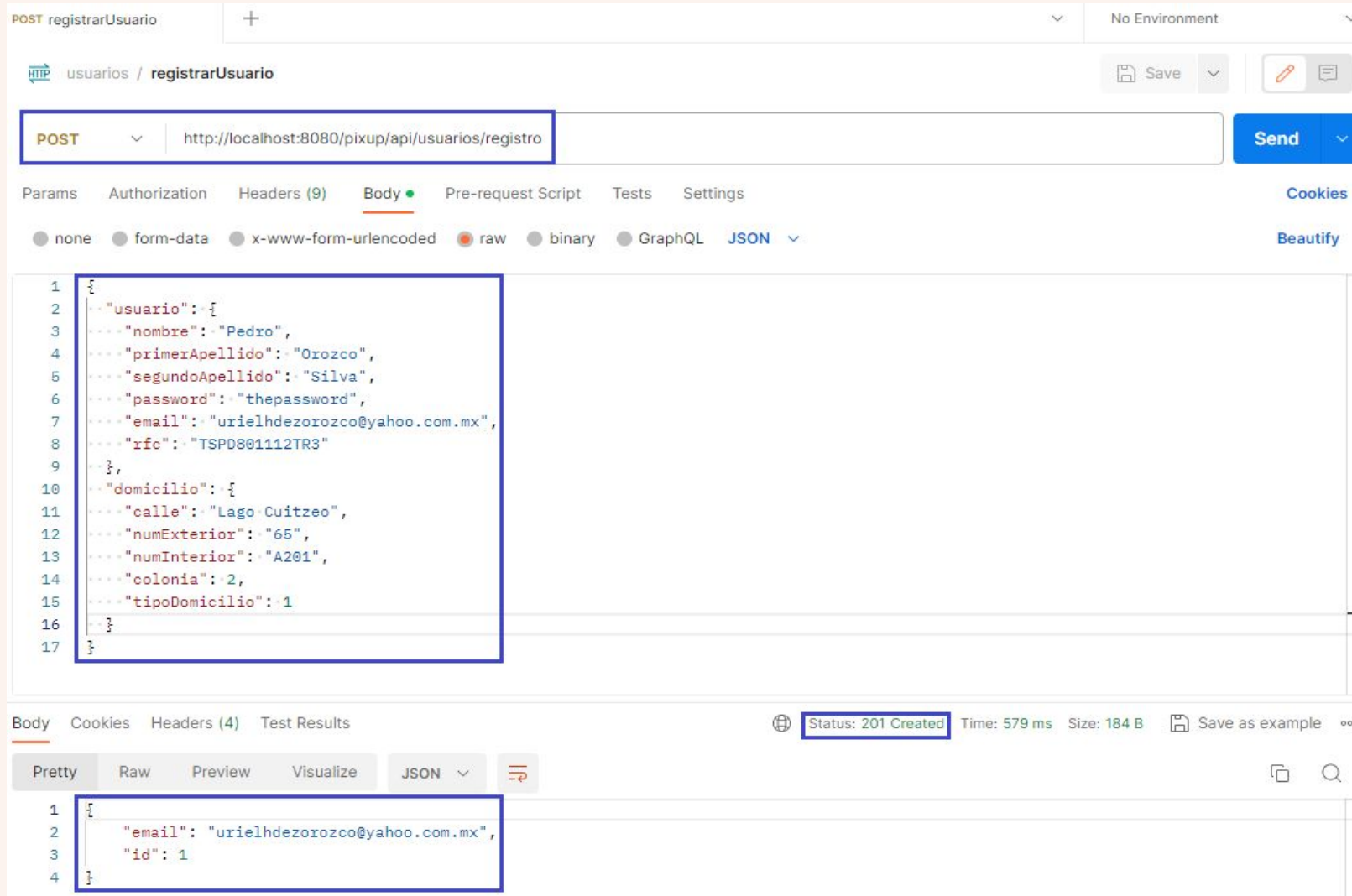
}
```

ExceptionHandler

```
@Provider
public class UsuarioAlreadyExistsExceptionHandler
    implements ExceptionMapper<UsuarioAlreadyExistsException> {

    @Override
    public Response toResponse(UsuarioAlreadyExistsException e) {
        return Response
            .status(Response.Status.CONFLICT)
            .entity(new ErrorResponse(
                Response.Status.CONFLICT.getStatusCode(),
                "BUSINESS_RULE",
                e.getMessage()))
            .build();
    }
}
```

Postman: UsuarioResource POST registrarUsuario



The screenshot displays the Postman interface for a POST request named "registrarUsuario". The URL is `http://localhost:8080/pixup/api/usuarios/registro`. The request body is a JSON object representing a user and their address, with line numbers 1 through 17 visible on the left. The response status is "201 Created", with a time of 579 ms and a size of 184 B. The response body is also shown in JSON format with line numbers 1 through 4.

Request URL: `http://localhost:8080/pixup/api/usuarios/registro`

Request Method: POST

Request Body (JSON):

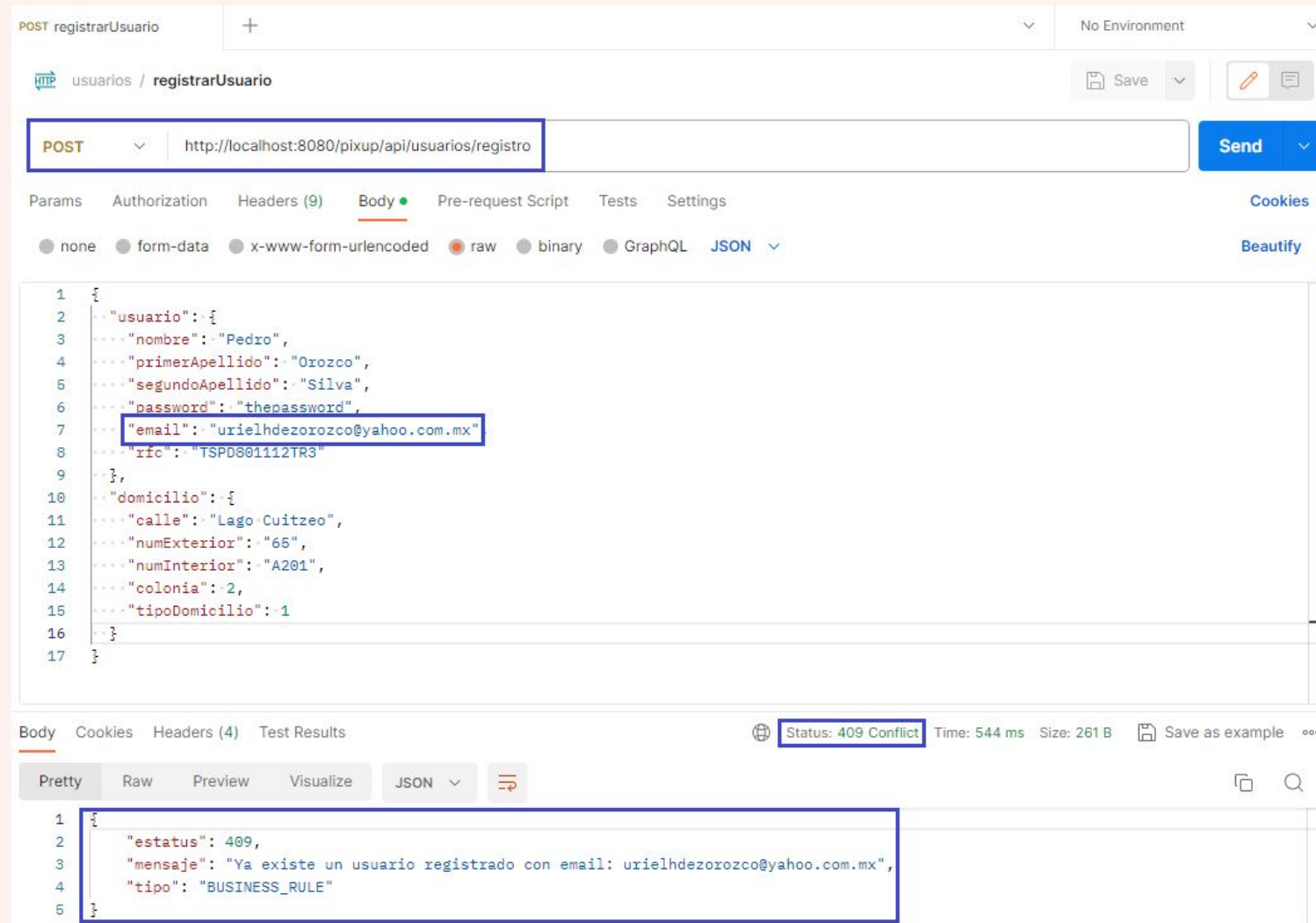
```
1 {
2   "usuario": {
3     "nombre": "Pedro",
4     "primerApellido": "Orozco",
5     "segundoApellido": "Silva",
6     "password": "thepassword",
7     "email": "urielhdezorozco@yahoo.com.mx",
8     "rfc": "TSPD801112TR3"
9   },
10  "domicilio": {
11    "calle": "Lago Cuitzeo",
12    "numExterior": "65",
13    "numInterior": "A201",
14    "colonia": "2",
15    "tipoDomicilio": 1
16  }
17 }
```

Response Status: 201 Created

Response Body (JSON):

```
1 {
2   "email": "urielhdezorozco@yahoo.com.mx",
3   "id": 1
4 }
```

Postman: UsuarioResource POST registrarUsuario - UK duplicated



POST registrarUsuario

usuarios / registrarUsuario

POST http://localhost:8080/pixup/api/usuarios/registro

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "usuario": {
3     "nombre": "Pedro",
4     "primerApellido": "Orozco",
5     "segundoApellido": "Silva",
6     "password": "thepassword",
7     "email": "urielhdezorozco@yahoo.com.mx",
8     "rfc": "TSPD801112TR3"
9   },
10  "domicilio": {
11    "calle": "Lago Cuitzeo",
12    "numExterior": "65",
13    "numInterior": "A201",
14    "colonia": "2",
15    "tipoDomicilio": "1"
16  }
17 }
```

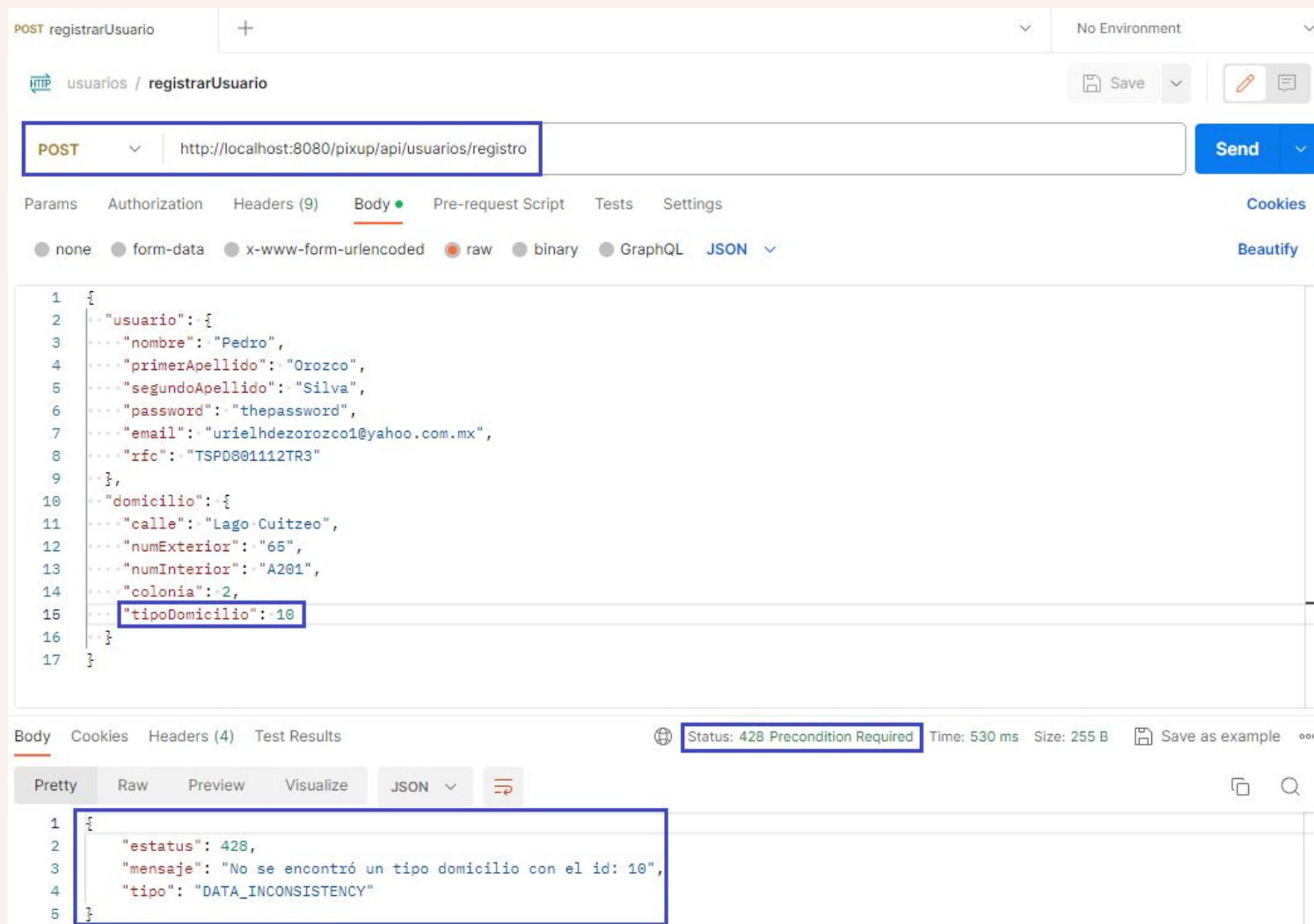
Body Cookies Headers (4) Test Results

Status: 409 Conflict Time: 544 ms Size: 261 B Save as example

Pretty Raw Preview Visualize JSON

```
1 {
2   "estatus": 409,
3   "mensaje": "Ya existe un usuario registrado con email: urielhdezorozco@yahoo.com.mx",
4   "tipo": "BUSINESS_RULE"
5 }
```

Postman: UsuarioResource POST registrarUsuario - TipoDomicilio inexistente



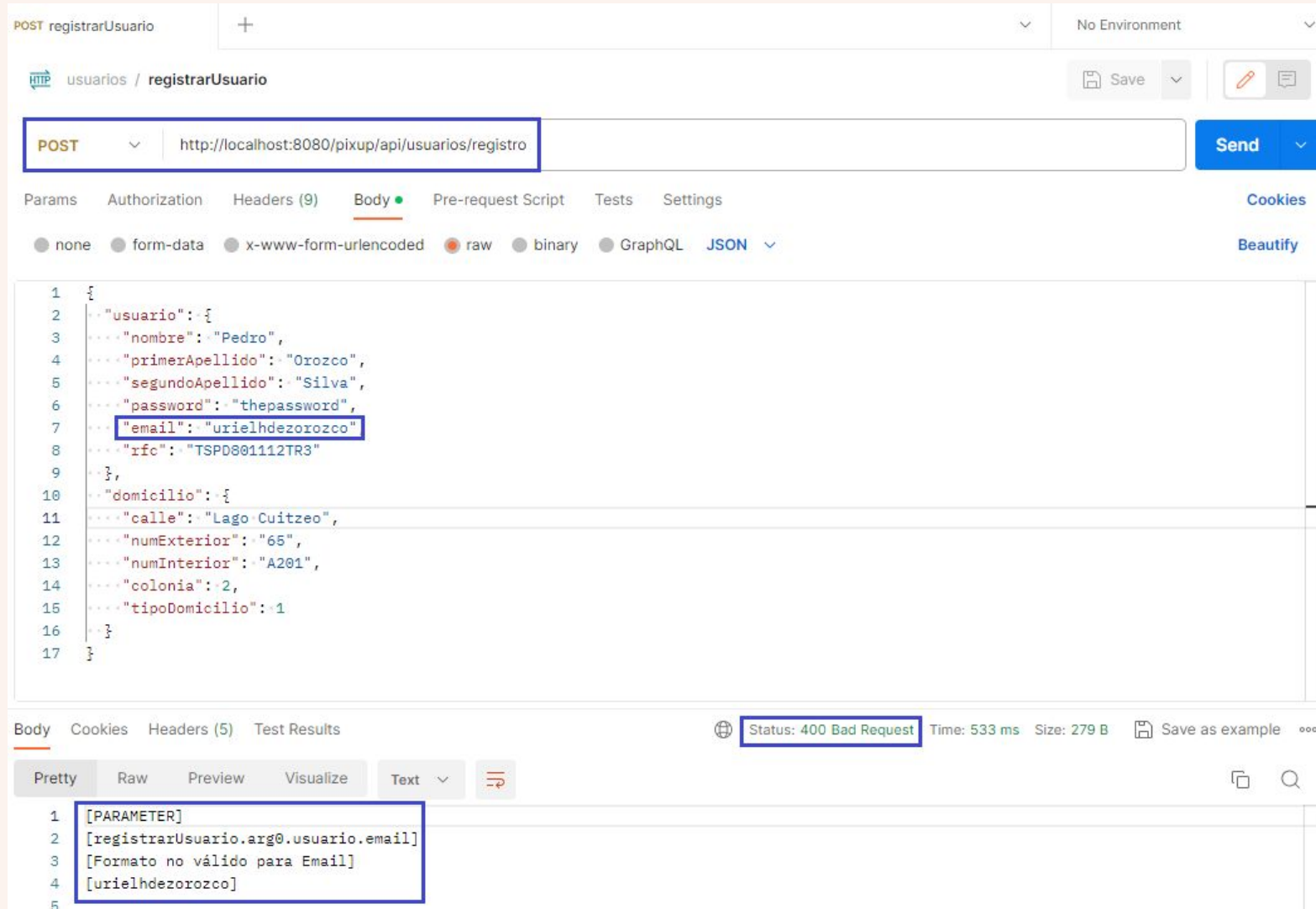
The screenshot displays a Postman interface for a POST request named 'registrarUsuario' to the endpoint 'http://localhost:8080/pixup/api/usuarios/registro'. The request body is a JSON object with user and address details. The 'tipoDomicilio' field is set to 10, which is highlighted with a red box. The response status is '428 Precondition Required', also highlighted with a red box. The response body, shown in the 'Body' tab, indicates an error: 'No se encontró un tipo domicilio con el id: 10'.

```
1 {
2   "usuario": {
3     "nombre": "Pedro",
4     "primerApellido": "Orozco",
5     "segundoApellido": "Silva",
6     "password": "thepassword",
7     "email": "urielhdezorozco1@yahoo.com.mx",
8     "rfc": "TSPD0801112TR3"
9   },
10  "domicilio": {
11    "calle": "Lago Cuitzeo",
12    "numExterior": "65",
13    "numInterior": "A201",
14    "colonia": "2",
15    "tipoDomicilio": 10
16  }
17 }
```

Status: 428 Precondition Required Time: 530 ms Size: 255 B Save as example

```
1 {
2   "estatus": 428,
3   "mensaje": "No se encontró un tipo domicilio con el id: 10",
4   "tipo": "DATA_INCONSISTENCY"
5 }
```

Postman: UsuarioResource POST registrarUsuario - Validator



POST registrarUsuario

usuarios / registrarUsuario

POST http://localhost:8080/pixup/api/usuarios/registro

Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "usuario": {
3     "nombre": "Pedro",
4     "primerApellido": "Orozco",
5     "segundoApellido": "Silva",
6     "password": "thepassword",
7     "email": "urielhdezorozco",
8     "rfc": "TSPD801112TR3"
9   },
10  "domicilio": {
11    "calle": "Lago Cuitzeo",
12    "numExterior": "65",
13    "numInterior": "A201",
14    "colonia": 2,
15    "tipoDomicilio": 1
16  }
17 }
```

Body Cookies Headers (5) Test Results

Status: 400 Bad Request Time: 533 ms Size: 279 B Save as example

Pretty Raw Preview Visualize Text

```
1 [PARAMETER]
2 [registrarUsuario.arg@usuario.email]
3 [Formato no válido para Email]
4 [urielhdezorozco]
5
```



Práctica Final

Contacto

Uriel Hernández
Solution Architect

urielhdezorozco@gmail.com

Redes sociales:

<https://www.linkedin.com/in/juho-mex>